Checkpoint Metacognitivo – Fase APLICA

¿Puedo implementar eliminarNodo() sin mirar el pseudocódigo? ¿Qué parte me cuesta más?

Sí, puedo implementarlo sin ver el pseudocódigo porque ya entiendo la lógica estructural:

Para eliminar un nodo conocido en el caso general debo actualizar:

nodo.anterior.siguiente = nodo.siguiente

nodo.siguiente.anterior = nodo.anterior

Sin embargo, la parte que más cuidado requiere (y puede costar más) son los casos frontera:

Cuando el nodo es la cabeza
Cuando el nodo es la cola
Cuando la lista tiene un solo elemento
Lo más delicado no es el caso general sino asegurar que:
cabeza y cola se actualicen correctamente
No queden referencias colgantes
Se mantenga la simetría bidireccional
Ahí es donde suelen aparecer los bugs sutiles.

¿Cuántos punteros debo actualizar al insertar en una lista doble vs. una simple? ¿Puedo listarlos?

Sí.

En lista simple :

Normalmente solo 1 puntero del nuevo nodo + 1 de la cabeza:

nuevo.siguiente = cabeza

cabeza = nuevo

Total: 2 actualizaciones principales

En lista doble:

Debo actualizar:

nuevo.siguiente = cabeza

cabeza.anterior = nuevo

cabeza = nuevo

Total: 3 actualizaciones

En lista doble (insertar al final):

cola.siguiente = nuevo

nuevo.anterior = cola
cola = nuevo

También 3 actualizaciones

Conclusión:

Lista simple → menos punteros, menos riesgo de errores.

Lista doble → más punteros que mantener sincronizados.

Si olvido uno, rompo la simetría.

Si la IA me dio código, ¿lo entiendo línea por línea o hay partes que "confío" sin verificar?

En este caso, sí lo entiendo línea por línea porque:

Sé por qué se actualiza cada puntero.

Entiendo qué pasa si quito una línea.

Puedo explicar la consecuencia de no actualizar anterior o siguiente.

Antes podía confiar en que "si compila, funciona".

Ahora entiendo que en estructuras con punteros eso no es suficiente: puede

compilar y aun así estar lógicamente mal.

Si hubiera una parte que podría generar duda sería:
El manejo del puntero actual en la Playlist.

La actualización del tamaño en eliminaciones intermedias.

Pero en general, ya no estoy copiando código: estoy entendiendo la estructura.

Checkpoint Metacognitivo – Fase REFLEXIONA

¿Puedo explicar a un compañero cuándo elegir lista doble vs. simple vs. arreglo, con ejemplos concretos?

Sí.

Arreglo (Array)

Lo elegiría cuando:

Necesito acceso rápido por índice → O(1)

El tamaño no cambia mucho

No hago muchas inserciones en medio

Ejemplo:

Lista de calificaciones.

Catálogo fijo de productos.

Acceso directo al elemento 25 sin recorrer.

Porque en un arreglo puedo hacer:

Elemento 25 Sin recorrer nada.

Lista simple

La elegiría cuando:

Solo necesito recorrer hacia adelante.

Inserto mucho al inicio.

No necesito retroceder.

Ejemplo:

Registro de eventos en orden.

Cola básica.

Historial donde solo se avanza.

Es más simple, menos punteros, menos riesgo de errores.

Lista doble

La elegiría cuando:

Necesito moverme adelante y atrás.

Eliminar nodos conocidos en O(1).

Mantener referencias a cabeza y cola.

Ejemplo concreto:

Playlist (como hicimos).

Botón atrás/adelante de un navegador.

Función undo/redo.

Historial de navegación en biblioteca digital.

Aquí la ventaja clave es poder hacer:

actual = actual.anterior

Eso en lista simple no existe.

¿Qué patrón descubrí al implementar? ¿Hay una "receta" para las operaciones de lista doble?

Sí, descubrí un patrón muy claro:
Patrón: "Actualizar en espejo"

Cada vez que conecto un nodo:

Si hago:

A.siguiente = B

Debo asegurar que:

B.anterior = A

Es una regla casi automática.

La receta general para insertar:

Crear el nodo

Conectar el nuevo nodo con sus vecinos.

Conectar los vecinos con el nuevo nodo.

Actualizar cabeza o cola si aplica.

Actualizar tamaño.

La receta general para eliminar:
Identificar caso:

cabeza

cola

nodo intermedio

único nodo

Reconectar anterior y siguiente.

Actualizar cabeza/cola si es necesario.

Mantener invariantes.

Actualizar tamaño.

Patrón importante que noté:

Casi todos los errores vienen de:

Olvidar actualizar un puntero.

No manejar casos frontera.

Romper la simetría bidireccional.

Si tuviera que enseñar "listas dobles" a alguien, ¿por dónde empezaría y qué diagrama dibujaría primero?

No empezaría con código.

Empezaría con un diagrama visual muy simple:

null ← A ⇄ B ⇄ C → null

↑ ↑

cabeza cola

Primero explicaría:

Cada nodo tiene dos manos:

una hacia adelante

una hacia atrás

Si A apunta a B, B debe apuntar a A.

Después mostraría:

Insertar al inicio

Insertar al final

Eliminar un nodo del medio

Y haría énfasis en:

"La clave no es el código, es mantener la simetría."

Solo después pasaría al pseudocódigo.